

TypeScript - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

TYPESCRIPT — MASTER NOTES 5

A Complete Reference Book for Interviews, Production Work & System Understanding 5

Written for someone who has never used TypeScript before — full setup steps, and how to add it on top of existing JavaScript/React code 5

TOPIC 1: TYPES 5

1. Introduction	5
1.1 What is TypeScript, and why does it exist?	5
1.2 The Exact Problem It Solves — Recall Java's Static Typing	5
2. FULL STEPS: Adding TypeScript to an EXISTING JavaScript/React Project	5
STEP 1 — Install TypeScript	5
STEP 2 — Create a tsconfig.json (TypeScript's Configuration File)	6
STEP 3 — Rename Your Files	6
STEP 4 — Start Adding Types (Where the Real Work Begins)	6
3. The Core Primitive Types	6
4. Type Inference — TypeScript Often Figures Out Types WITHOUT Explicit Annotations	6
5. Real Life Analogy	7
6. Edge Cases	7
7. Common Mistakes	7
8. Frequently Asked Interview Questions	7

TOPIC 2: INTERFACES 8

1. Introduction	8
2. Code Example — Defining and Using an Interface	8
3. Interfaces for Function Shapes	8
4. Extending Interfaces (Like Java's extends)	9
5. interface vs type — A Frequently Asked Distinction	9
6. Real Life Analogy	9
7. Edge Cases	9
8. Frequently Asked Interview Questions	9

TOPIC 3: GENERICS	10
1. Introduction	10
2. Code Example — Direct Comparison to Java	10
3. Generic Functions — A Real, Practical Use Case	10
4. Generic Interfaces — Modeling an API Response (A Very Common Real Pattern)	11
5. Bounded Generics (Recap From Java, Same Syntax Concept)	11
6. Real Life Analogy	11
7. Frequently Asked Interview Questions	11
TOPIC 4: ENUMS	11
1. Introduction	12
2. Code Example	12
3. String Enums — Usually the Better Choice (Same Reasoning as Java's EnumType.STRING)	12
4. A Modern Alternative — "Union of String Literals" (Increasingly Preferred in the React Community)	12
5. Real Life Analogy	12
6. Frequently Asked Interview Questions	12
TOPIC 5: UTILITY TYPES	13
1. Introduction	13
2. Code Examples — The Most Commonly Used Utility Types	13
3. Real Life Analogy	14
4. Frequently Asked Interview Questions	14
TOPIC 6: TYPE NARROWING	14
1. Introduction	14
2. Code Example — typeof Narrowing	14
3. Narrowing With a Custom "Type Guard" Function	15
4. Real Life Analogy	15
5. Frequently Asked Interview Questions	15
TOPIC 7: MODULES	15
1. Introduction	15
2. Code Example	16

3. Frequently Asked Interview Questions	16
---	----

TOPIC 8: REACT WITH TYPESCRIPT 16

1. Introduction	16
2. Typing Props	17
3. Typing useState	17
4. Typing Event Handlers	17
5. Typing Children and Component Return Types	18
6. Real Life Analogy	18
7. Common Mistakes	18
8. Best Practices	18
9. Frequently Asked Interview Questions	18
10. Revision Notes (Entire TypeScript Section Consolidated)	19
11. Homework (Covers Entire TypeScript Section)	19
FINAL SUMMARY — TYPESCRIPT COMPLETE	19

TYPESCRIPT — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

Written for someone who has never used TypeScript before — full setup steps, and how to add it on top of existing JavaScript/React code

TOPIC 1: TYPES

1. Introduction

1.1 What is TypeScript, and why does it exist?

TypeScript is a superset of JavaScript (created by Microsoft) that adds STATIC TYPING on top of it — you write TypeScript (`.ts/.tsx` files), and a COMPILER translates it into plain JavaScript (`.js`) that runs in the browser exactly as before. Every valid JavaScript file is ALREADY (mostly) valid TypeScript — TypeScript is purely an ADDITIONAL LAYER, not a replacement language.

1.2 The Exact Problem It Solves — Recall Java's Static Typing

You already know EXACTLY why this matters, from Core Java: Java catches type errors at COMPILE TIME (`int x = "hello";` fails immediately). Plain JavaScript has NO such protection — it's DYNAMICALLY typed, meaning type errors only surface at RUNTIME, often in production, potentially far from where the actual mistake was made:

```
// PLAIN JAVASCRIPT - compiles/runs FINE, but crashes at RUNTIME:
function calculateBonus(salary) {
  return salary * 0.1;
}
calculateBonus("75000"); // "75000" * 0.1 -> actually works due to JS's type coercion... but:
calculateBonus({ amount: 75000 }); // returns NaN - a SILENT bug, no error at all, just wrong data!

// TYPESCRIPT - this EXACT mistake is caught IMMEDIATELY, at COMPILE TIME, before you ever run it:
function calculateBonus(salary: number): number {
  return salary * 0.1;
}
calculateBonus({ amount: 75000 });
// COMPILER ERROR: Argument of type '{ amount: number; }' is not assignable to parameter of type 'number'.
```

This is EXACTLY the same value proposition Java's type system gives you over an untyped language — TypeScript brings that SAME safety net to the JavaScript/React world.

2. FULL STEPS: Adding TypeScript to an EXISTING JavaScript/React Project

STEP 1 — Install TypeScript

```
npm install --save-dev typescript @types/react @types/react-dom @types/node
```

`@types/react` provides TYPE DEFINITIONS for the React library itself (React is written in plain JS, so these separate "type definition" packages describe its shapes for TypeScript's benefit) — this pattern (a separate `@types/` package) is common for many JS libraries.

STEP 2 — Create a `tsconfig.json` (TypeScript's Configuration File)

```
npx tsc --init

{
  "compilerOptions": {
    "target": "ES2020",
    "lib": ["DOM", "ES2020"],
    "jsx": "react-jsx",
    "strict": true, // enables ALL strict type-checking options - ALWAYS recommended
    "module": "ESNext",
    "moduleResolution": "bundler",
    "esModuleInterop": true,
    "skipLibCheck": true,
    "noEmit": true // in most React setups (Vite/Create React App), a separate BUNDLER
  } // actually compiles the code; tsc is used just for TYPE CHECKING
}
```

STEP 3 — Rename Your Files

`Component.js` -> `Component.tsx` (".tsx" for files containing JSX)
`utils.js` -> `utils.ts` (".ts" for plain TypeScript, no JSX)

This is the KEY practical step: TypeScript is INCREMENTALLY ADOPTABLE — you can rename files ONE AT A TIME, and even leave PLAIN `.js` files sitting alongside `.ts` files in the SAME project (with a permissive config) — you don't need to convert an entire large existing codebase all at once.

STEP 4 — Start Adding Types (Where the Real Work Begins)

```
// BEFORE (plain JavaScript):
function greet(name) {
  return "Hello, " + name;
}

// AFTER (TypeScript) - just ADD a type annotation after each parameter and the return type:
function greet(name: string): string {
  return "Hello, " + name;
}
```

3. The Core Primitive Types

```
let age: number = 30;
let name: string = "Alice";
let isActive: boolean = true;
let tags: string[] = ["premium", "verified"]; // an ARRAY of strings
let coordinates: [number, number] = [10, 20]; // a TUPLE - a FIXED-length array with SPECIFIC types per position
let anything: any = "avoid this type when possible"; // opts OUT of type checking entirely - defeats the purpose!
let notSure: unknown = fetchSomeData(); // SAFER than 'any' - forces a type CHECK before use
```

4. Type Inference — TypeScript Often Figures Out Types WITHOUT Explicit Annotations

```
let salary = 75000; // TypeScript INFERS this is a 'number' automatically - no ": number" needed!
salary = "75000"; // COMPILE ERROR - TypeScript remembers the INFERRED type and still enforces it
```

You don't need to annotate EVERY single variable — TypeScript is smart enough to INFER types from the assigned value in most cases, only requiring EXPLICIT annotations for function PARAMETERS (which have no initial value to infer from) and a few other specific cases.

5. Real Life Analogy

- **Restaurant:** Plain JavaScript is like a kitchen that will ATTEMPT to cook with WHATEVER ingredient you hand it, even if it's completely wrong for the recipe (silently producing a bad dish, or `NaN`). TypeScript is like a kitchen QUALITY-CONTROL step that checks EVERY ingredient against the recipe's requirements the MOMENT you hand it over — refusing to even START cooking if you tried to hand over, say, a shoe instead of a potato.

6. Edge Cases

Case	Result
Using the <code>any</code> type	COMPLETELY disables type checking for that value — technically valid TypeScript, but defeats the entire purpose; avoid it except as a genuine, temporary last resort
TypeScript compile errors	By default, DO NOT prevent the code from actually running in many dev setups (the bundler may still emit JS) — but in a properly configured CI/CD pipeline (tying back to the CI/CD section), a failing <code>tsc</code> type-check should BLOCK the build/deployment

7. Common Mistakes

- Overusing `any` to "make errors go away" instead of properly typing the actual value — this silently reintroduces plain JavaScript's exact original problem.
- Trying to convert an ENTIRE large existing codebase to TypeScript all at once instead of incrementally, file by file.

8. Frequently Asked Interview Questions

- 1 **What is TypeScript?** A superset of JavaScript adding static typing, compiled down to plain JavaScript that runs identically in any browser/Node.js environment.
- 2 **What EXACT problem does TypeScript solve, that you already understand from Java?** It catches TYPE ERRORS at COMPILE TIME rather than at runtime — the same value proposition as Java's static typing over a dynamically-typed language.
- 3 **Can you add TypeScript to an existing JavaScript project incrementally?** Yes — files can be renamed and typed one at a time (`.js` → `.ts/.tsx`), and plain JavaScript files can coexist alongside TypeScript files in the same project.
- 4 **Why should you avoid using the `any` type?** It completely disables type checking for that value, silently reintroducing the exact runtime-error risk TypeScript exists to prevent.

- 5 **What is type inference?** TypeScript automatically determining a variable's type from its assigned value, without requiring an explicit type annotation.
-
-

TOPIC 2: INTERFACES

1. Introduction

What is it? A TypeScript `interface` defines the SHAPE an object must have — WHICH properties it must contain, and their types — directly analogous to a Java interface defining a CONTRACT, though TypeScript interfaces describe DATA SHAPES far more often than they describe BEHAVIOR/methods (unlike Java, where interfaces are primarily about behavior contracts).

2. Code Example — Defining and Using an Interface

```
interface Employee {
  id: number;
  firstName: string;
  lastName: string;
  salary: number;
  department?: string; // the '?' marks an OPTIONAL property - may or may not be present
  readonly email: string; // 'readonly' - can be SET once, but never REASSIGNED afterward (like Java's final)
}

function printEmployee(employee: Employee): void {
  console.log(`${employee.firstName} ${employee.lastName}: $$${employee.salary}`);
}

const alice: Employee = {
  id: 1,
  firstName: "Alice",
  lastName: "Johnson",
  salary: 75000,
  email: "alice@company.com",
  // 'department' omitted - fine, since it's OPTIONAL
};

printEmployee(alice);

alice.email = "new@company.com"; // COMPILER ERROR - 'email' is readonly, cannot be reassigned!
```

3. Interfaces for Function Shapes

```
interface CalculateBonus {
  (salary: number, percentage: number): number; // defines a FUNCTION'S parameter/return shape
}

const calculateBonus: CalculateBonus = (salary, percentage) => salary * (percentage / 100);
```

4. Extending Interfaces (Like Java's `extends`)

```
interface Person {
  name: string;
  age: number;
}

interface Employee extends Person { // Employee has EVERYTHING Person has, PLUS these additional fields
  salary: number;
  department: string;
}

const emp: Employee = { name: "Bob", age: 30, salary: 60000, department: "Sales" };
```

5. `interface` vs `type` — A Frequently Asked Distinction

```
interface Employee { name: string; salary: number; } // interface - can be EXTENDED, can be "re-opened" (merged)
type Employee = { name: string; salary: number; }; // type alias - CANNOT be re-opened, but supports // UNIONS and other more advanced type expressions
```

Aspect	Interface	Type
Extending	<code>extends</code> keyword	<code>&</code> (intersection) operator
Can represent unions ("A" "B")?	No	Yes
Community convention	Preferred for OBJECT SHAPES (like a DTO)	Preferred for unions, primitives, complex type expressions

6. Real Life Analogy

- **Bank:** An interface is like a STANDARDIZED account-opening FORM TEMPLATE — it specifies EXACTLY which fields are REQUIRED (name, account type) and which are OPTIONAL (a middle name), and the bank's system REJECTS any submitted form missing a required field or containing a field of the wrong type, exactly like TypeScript rejecting an object that doesn't match its declared interface shape.

7. Edge Cases

Case	Result
Providing an object with an EXTRA property NOT declared in the interface	Usually a compile ERROR for object LITERALS assigned directly ("excess property checking"), but NOT for a variable assigned first, then passed — a genuinely subtle TypeScript nuance
Omitting a REQUIRED (non-optional, no <code>?</code>) property	Compile-time error — the object doesn't satisfy the interface

8. Frequently Asked Interview Questions

- 1 **What does a TypeScript interface define?** The required SHAPE of an object — which properties it must have, their types, and which are optional.

- 2 **What does the ? after a property name mean?** That property is OPTIONAL — the object can be valid with or without it.
- 3 **What does readonly do on an interface property?** Allows the value to be SET once (e.g., at object creation) but never REASSIGNED afterward — similar in spirit to Java's `final`.
- 4 **When would you prefer interface over type (or vice versa)?** interface is generally preferred for defining OBJECT SHAPES (can be extended/merged); type is needed for UNION types and other type expressions an interface cannot express.

TOPIC 3: GENERICS

1. Introduction

What is it? TypeScript Generics let you write REUSABLE code that works with MULTIPLE types WHILE STILL preserving type safety — CONCEPTUALLY IDENTICAL to Java Generics (Core Java's Generics topic), solving the EXACT same problem: avoiding either code duplication (one version per type) OR losing type safety entirely (using any everywhere).

2. Code Example — Direct Comparison to Java

```
// JAVA Generics (you already know this):
class Box<T> {
  private T content;
  public void set(T content) { this.content = content; }
  public T get() { return content; }
}

// TypeScript Generics - the SAME concept, nearly identical syntax:
class Box<T> {
  private content: T;
  set(content: T): void { this.content = content; }
  get(): T { return this.content; }
}

const numberBox = new Box<number>();
numberBox.set(42);
const value: number = numberBox.get(); // TypeScript KNOWS this is a number, no cast needed
```

3. Generic Functions — A Real, Practical Use Case

```
function getFirstElement<T>(array: T[]): T {
  return array[0];
}

const firstNumber = getFirstElement<number>([1, 2, 3]); // T is explicitly number
const firstName = getFirstElement(["Alice", "Bob"]); // T is INFERRED as string automatically!
```

4. Generic Interfaces — Modeling an API Response (A Very Common Real Pattern)

```
interface ApiResponse<T> {
  data: T;
  success: boolean;
  errorMessage?: string;
}

interface Employee { id: number; name: string; }

async function fetchEmployee(id: number): Promise<ApiResponse<Employee>> {
  const response = await fetch(`http://localhost:8080/api/employees/${id}`);
  const employee: Employee = await response.json();
  return { data: employee, success: true };
}
```

5. Bounded Generics (Recap From Java, Same Syntax Concept)

```
interface HasId { id: number; }

function findById<T extends HasId>(items: T[], id: number): T | undefined { // T MUST have an 'id' property
  return items.find(item => item.id === id);
}
```

6. Real Life Analogy

- **Restaurant:** A generic `Box<T>` is like a STANDARDIZED, reusable storage container design that can hold EITHER vegetables OR drinks OR desserts — the SAME container blueprint is reused, but each specific instance is LABELED and RESTRICTED to hold ONLY its designated item type, exactly like the Java Generics analogy from Core Java.

7. Frequently Asked Interview Questions

- 1 **What are TypeScript Generics, and how do they compare to Java Generics?** A way to write reusable code/types that work across multiple types while preserving type safety — conceptually and syntactically very similar to Java's `<T>` generic type parameters.
- 2 **Give a common real-world use case for a generic interface.** Modeling a standardized API response wrapper (`ApiResponse<T>`) that can hold ANY specific data type (`Employee`, `Department`) while keeping that specific type information intact.
- 3 **What does `T extends HasId` mean in a generic function signature?** A BOUNDED generic — `T` can be any type, as long as it has (at minimum) the properties defined by `HasId`, allowing the function to safely access `.id` on values of type `T`.

TOPIC 4: ENUMS

1. Introduction

What is it? TypeScript Enums let you define a set of NAMED CONSTANTS — directly analogous to Java's `enum` (Core Java's Enums topic), providing the SAME type-safety benefit over using raw strings/numbers scattered throughout your code.

2. Code Example

```
enum EmployeeStatus {  
  ACTIVE, // 0 (numeric enums auto-increment from 0 by default)  
  ON_LEAVE, // 1  
  TERMINATED, // 2  
}  
  
let status: EmployeeStatus = EmployeeStatus.ACTIVE;  
console.log(status); // 0
```

3. String Enums — Usually the Better Choice (Same Reasoning as Java's `EnumType.STRING`)

```
enum EmployeeStatus {  
  ACTIVE = "ACTIVE", // explicit STRING values - readable in logs/JSON, doesn't silently  
  ON_LEAVE = "ON_LEAVE", // shift meaning if reordered (exactly the Java EnumType.ORDINAL warning!)  
  TERMINATED = "TERMINATED",  
}  
  
const status: EmployeeStatus = EmployeeStatus.ACTIVE;  
console.log(status); // "ACTIVE" - readable, and safe from reordering issues
```

4. A Modern Alternative — "Union of String Literals" (Increasingly Preferred in the React Community)

```
type EmployeeStatus = "ACTIVE" | "ON_LEAVE" | "TERMINATED"; // NOT a real enum, but achieves a similar  
effect  
  
function updateStatus(status: EmployeeStatus) { /* ... */ }  
updateStatus("ACTIVE"); // valid  
updateStatus("RESIGNED"); // COMPILER ERROR - "RESIGNED" isn't one of the allowed literal values
```

Many modern TypeScript/React style guides PREFER this "string literal union" pattern over `enum` — it requires NO extra runtime code (an `enum` actually generates real JavaScript objects at compile time, while a union type is a PURELY compile-time-only construct, adding zero runtime overhead), and integrates more naturally with plain JSON data from a REST API.

5. Real Life Analogy

- **Restaurant:** An Enum (or string-literal union) is like a MENU'S predefined list of valid order STATUSES ("Received," "Preparing," "Ready") — the kitchen SYSTEM will only accept EXACTLY one of these predefined values, rejecting any typo'd or invalid status immediately, exactly like Java's enum type-safety over raw strings.

6. Frequently Asked Interview Questions

- 1 **What does a TypeScript `enum` do, conceptually compared to Java's `enum`?** Defines a fixed set of named constants, providing type safety over raw string/number "magic values" — the same core purpose as Java's `enum`.
 - 2 **Why prefer a string enum over a numeric enum?** Same reasoning as Java's `EnumType.STRING` over `EnumType.ORDINAL` — readable values that don't silently shift meaning if reordered, unlike numeric enums which auto-increment based on declaration order.
 - 3 **What's an increasingly popular alternative to `enum` in modern TypeScript/React code?** A "string literal union" type (type `Status = "ACTIVE" | "ON_LEAVE"`), which adds zero runtime overhead (purely compile-time) unlike a real `enum`, which generates actual JavaScript objects.
-

TOPIC 5: UTILITY TYPES

1. Introduction

What is it? Utility Types are BUILT-IN TypeScript helpers that TRANSFORM an existing type into a NEW, related type — avoiding the need to manually rewrite a nearly-identical interface for common variations (e.g., "the same shape, but every field optional").

2. Code Examples — The Most Commonly Used Utility Types

```
interface Employee {
  id: number;
  firstName: string;
  lastName: string;
  salary: number;
}

// Partial<T> - makes EVERY property OPTIONAL - perfect for a PATCH/update DTO (recall REST APIs'
// PATCH topic!)
type EmployeeUpdate = Partial<Employee>;
function updateEmployee(id: number, changes: EmployeeUpdate) { /* only the PROVIDED fields need to be
present */ }
updateEmployee(1, { salary: 80000 }); // valid - firstName/lastName/id are all optional here

// Pick<T, Keys> - selects ONLY specific properties from an existing type
type EmployeeSummary = Pick<Employee, "id" | "firstName">;
const summary: EmployeeSummary = { id: 1, firstName: "Alice" };

// Omit<T, Keys> - takes EVERYTHING EXCEPT the specified properties
type EmployeeWithoutSalary = Omit<Employee, "salary">;

// Readonly<T> - makes EVERY property readonly (immutable after creation)
const frozenEmployee: Readonly<Employee> = { id: 1, firstName: "Alice", lastName: "J", salary: 75000
};
frozenEmployee.salary = 80000; // COMPILER ERROR - readonly!

// Record<Keys, ValueType> - a MAP/object type with SPECIFIC key and value types
type DepartmentHeadcount = Record<string, number>;
const headcount: DepartmentHeadcount = { Engineering: 42, Sales: 18 };
```

3. Real Life Analogy

- **Bank:** `Partial<Employee>` is like a bank's "UPDATE ONLY WHAT CHANGED" form — you're allowed to fill in ONLY the specific fields you actually want to change, unlike the FULL, all-fields-required original account-opening form (`Employee` itself). `Pick/Omit` are like producing a SIMPLIFIED, PUBLIC summary sheet showing only SOME fields from a much more detailed FULL internal record.

4. Frequently Asked Interview Questions

- 1 **What does `Partial<T>` do?** Creates a new type identical to `T`, but with EVERY property made optional — commonly used for update/PATCH-style DTOs.
 - 2 **What's the difference between `Pick<T, Keys>` and `Omit<T, Keys>`?** `Pick` selects ONLY the specified properties; `omit` selects EVERYTHING EXCEPT the specified properties.
 - 3 **What does `Readonly<T>` do?** Makes every property of `T` immutable after initial assignment.
 - 4 **What is `Record<K, V>` typically used for?** Defining a map/dictionary-like object type with specific key and value types, e.g., `Record<string, number>`.
-

TOPIC 6: TYPE NARROWING

1. Introduction

What is it? Type Narrowing is TypeScript's ability to REFINE a broader type into a MORE SPECIFIC one WITHIN a specific code branch, based on a runtime CHECK — directly analogous to Java 16+'s pattern matching for `instanceof` (Core Java's Type Casting topic).

2. Code Example — `typeof` Narrowing

```
function formatValue(value: string | number): string { // a UNION type - could be EITHER
  if (typeof value === "string") {
    return value.toUpperCase(); // TypeScript KNOWS 'value' is a string HERE - toUpperCase() is safe
  } else {
    return value.toFixed(2); // TypeScript KNOWS 'value' is a number HERE - toFixed() is safe
  }
}
```

3. Narrowing With a Custom "Type Guard" Function

```
interface Dog { bark: () => void; }
interface Cat { meow: () => void; }

function isDog(animal: Dog | Cat): animal is Dog { // a TYPE GUARD - a special return type telling
return (animal as Dog).bark !== undefined; // TypeScript what this function's boolean result MEANS
}

function makeSound(animal: Dog | Cat) {
if (isDog(animal)) {
animal.bark(); // TypeScript NARROWS to Dog inside this branch, based on the type guard!
} else {
animal.meow(); // narrowed to Cat here
}
}
```

4. Real Life Analogy

- **Bank:** Type Narrowing is like a bank teller who receives a GENERIC "customer request" (a union type — could be a deposit OR a withdrawal), CHECKS which SPECIFIC kind it actually is, and from THAT POINT ONWARD in handling it, treats it with the FULL, SPECIFIC procedure for exactly that request type — safely accessing whatever SPECIFIC details that particular request type provides, exactly like Java's `instanceof` pattern matching narrowing a general reference to its specific subtype.

5. Frequently Asked Interview Questions

- 1 **What is Type Narrowing?** TypeScript refining a broader (often union) type into a more specific one within a code branch, based on a runtime check like `typeof` or a custom type guard.
- 2 **What's the Java equivalent concept?** Pattern matching for `instanceof` (Java 16+), which similarly narrows a general reference type to a specific subtype within an `if` branch.
- 3 **What is a "type guard" function?** A custom function with a special `x is Type` return type annotation, telling TypeScript's narrowing logic exactly what a `true/false` result means for the checked value's type.

TOPIC 7: MODULES

1. Introduction

What is it? TypeScript uses the EXACT SAME `export/import` module system as modern JavaScript (JavaScript section, Topic 7) — TypeScript adds NOTHING new here syntactically; it simply TYPE-CHECKS across those module boundaries too, ensuring an imported function/value is used CORRECTLY according to its declared types wherever it's imported.

2. Code Example

```
// employeeService.ts
export interface Employee {
  id: number;
  firstName: string;
  salary: number;
}

export async function getEmployee(id: number): Promise<Employee> {
  const response = await fetch(`http://localhost:8080/api/employees/${id}`);
  return response.json();
}

// EmployeeProfile.tsx
import { getEmployee, Employee } from "./employeeService"; // importing BOTH a value AND a TYPE!

async function loadProfile(id: number) {
  const employee: Employee = await getEmployee(id); // fully TYPE-CHECKED across the file boundary
  console.log(employee.firstName);
}
```

3. Frequently Asked Interview Questions

- 1 **Does TypeScript use a different module system than JavaScript?** No — the exact same `export/import` syntax; TypeScript adds compile-time type-checking ACROSS module boundaries on top of it.
- 2 **Can you export/import a TYPE (like an interface), not just a value?** Yes — types and values can be exported/imported using the same syntax, often from the same file.

TOPIC 8: REACT WITH TYPESCRIPT

1. Introduction

This topic ties EVERYTHING together — converting the JavaScript React components from the React section into fully-typed TypeScript, the way you'd ACTUALLY write React code in most modern, professional codebases today.

2. Typing Props

```
// BEFORE (plain JavaScript, from the React section):
function EmployeeCard({ name, department, salary }) {
  return <div><h3>{name}</h3><p>{department}</p><p>${salary}</p></div>;
}

// AFTER (TypeScript) - define an interface for the props, exactly like an API request/response DTO:
interface EmployeeCardProps {
  name: string;
  department: string;
  salary: number;
}

function EmployeeCard({ name, department, salary }: EmployeeCardProps) {
  return <div><h3>{name}</h3><p>{department}</p><p>${salary}</p></div>;
}

// Using it WRONG is now caught at COMPILE TIME, not discovered at runtime:
<EmployeeCard name="Alice" department="Engineering" salary="75000" />
// COMPILE ERROR: Type 'string' is not assignable to type 'number'. (salary should be a number!)
```

3. Typing `useState`

```
import { useState } from "react";

interface Employee { id: number; firstName: string; salary: number; }

function EmployeeProfile() {
  const [employee, setEmployee] = useState<Employee | null>(null); // explicit generic - starts as null,
  // becomes an Employee later
  const [loading, setLoading] = useState<boolean>(true);
  const [count, setCount] = useState(0); // TypeScript INFERS 'number' automatically here - no <> needed

  useEffect(() => {
    fetch("http://localhost:8080/api/employees/1")
      .then(res => res.json())
      .then((data: Employee) => { // explicitly typing the resolved fetch data
        setEmployee(data);
        setLoading(false);
      });
  }, []);

  if (loading || !employee) return <p>Loading...</p>;
  return <h2>{employee.firstName}</h2>; // SAFE - TypeScript KNOWS 'employee' isn't null past this point
  // (narrowing!)
}
```

4. Typing Event Handlers

```
function SearchBox() {
  const [query, setQuery] = useState("");

  const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => { // the SPECIFIC React event
    type
    setQuery(event.target.value);
  };

  return <input value={query} onChange={handleChange} />;
}
```

5. Typing Children and Component Return Types

```
interface CardProps {
  children: React.ReactNode; // the CORRECT type for "anything React can render" (text, elements, etc.)
}

function Card({ children }: CardProps): React.JSX.Element {
  return <div className="card">{children}</div>;
}
```

6. Real Life Analogy

- **Restaurant:** Using plain JavaScript React components is like a kitchen accepting ANY handwritten order slip, hoping it's filled in correctly. Using TypeScript with React is like the kitchen having a STANDARDIZED, PRE-PRINTED order form with clearly labeled, TYPE-CHECKED fields — an order slip missing a required field, or putting text where a number belongs, is REJECTED immediately at the WINDOW (compile time), long before it ever reaches the cook (runtime).

7. Common Mistakes

- Not typing `useState`'s generic when the initial value doesn't fully convey the eventual type (e.g., starting with `null` but later expecting an `Employee` object) — leads to TypeScript inferring an overly narrow type.
- Using `any` for event handler parameters instead of the correct specific React event type (`React.ChangeEvent<HTMLInputElement>`, `React.FormEvent`, etc.), losing autocomplete and type safety on `event.target`.

8. Best Practices

- Define an `interface` for EVERY component's props — this is the single most valuable, most commonly applied TypeScript practice in React codebases.
- Explicitly type `useState`'s generic whenever the initial value (like `null`) doesn't fully represent the eventual type the state will hold.
- Use `React.ChangeEvent<HTMLInputElement>` (and similar specific types) for event handlers, rather than `any`.

9. Frequently Asked Interview Questions

- 1 **Why is defining a props `interface` considered the single most valuable TypeScript practice in React?** It catches incorrect usage of a component (wrong prop types, missing required props) at COMPILE TIME, across the ENTIRE codebase, rather than discovering the bug at runtime, potentially in production.
- 2 **How do you type `useState` when the initial value doesn't represent the eventual full type?** Provide an explicit generic type argument, e.g., `useState<Employee | null>(null)`.
- 3 **What TypeScript type represents "anything React can render" for a `children` prop?** `React.ReactNode`.
- 4 **How does TypeScript's narrowing help inside a component after a loading/null check?** Once you've checked `if (!employee) return ...`, TypeScript KNOWS `employee` cannot be `null` for the REST of that function, letting you safely access its properties without further checks.

10. Revision Notes (Entire TypeScript Section Consolidated)

TYPESCRIPT – CHEAT SHEET

=====

SOLVES: brings Java-style COMPILE-TIME type safety to JavaScript - a SUPERSET of JS, compiles down to plain JS, adoptable INCREMENTALLY (rename .js -> .ts/.tsx file by file)

SETUP: npm install --save-dev typescript @types/react @types/react-dom
npx tsc --init (tsconfig.json), strict: true recommended

TYPES: number/string/boolean/array/tuple, type INFERENCE (often no annotation needed), avoid 'any' (disables checking entirely)

INTERFACE: defines an object SHAPE (like a DTO) - ? for optional, readonly for immutable-after-set, extends for inheritance
interface vs type: interface for object shapes, type for unions

GENERIC: <T> - IDENTICAL concept/syntax to Java Generics - reusable, type-safe code across multiple types (Box<T>, ApiResponse<T>)

ENUMS: like Java enum - prefer STRING enums (same EnumType.STRING reasoning) OR modern "string literal union" types (zero runtime cost)

UTILITY TYPES: Partial<T> (all optional, great for PATCH DTOs), Pick<T,Keys> / Omit<T,Keys> (subset/exclude fields), Readonly<T>, Record<K,V> (typed map)

TYPE NARROWING: typeof / custom type guards (x is Type) refine a union type to a specific one within a branch - same idea as Java's instanceof pattern matching

MODULES: SAME export/import as JS - TypeScript just type-checks across the boundary; can export/import TYPES alongside values

REACT + TYPESCRIPT: interface for EVERY component's props (the #1 practice), useState<T>() explicit generic when needed, React.ChangeEvent<HTMLInputElement> for event handlers, React.ReactNode for children

11. Homework (Covers Entire TypeScript Section)

- 1 Add TypeScript to your React project from the React section's homework, converting EmployeeCard to a .tsx file with a proper EmployeeCardProps interface.
- 2 Type your useEmployee custom hook's return value and the useState calls inside it.
- 3 Define a generic ApiResponse<T> interface, and use it to type the response from your Spring Boot /api/employees endpoint.
- 4 Create a Partial<Employee> type for your update form, and verify TypeScript allows submitting only a subset of fields.

FINAL SUMMARY — TYPESCRIPT COMPLETE

This concludes the **TypeScript** section — 8 topics:

- 1 **Types** — the exact Java-parallel value proposition, full incremental setup steps.
- 2 **Interfaces** — object shapes, optional/readonly properties, extending.
- 3 **Generics** — the direct Java Generics parallel.
- 4 **Enums** — string enums vs the modern string-literal-union alternative.
- 5 **Utility Types** — `Partial/Pick/Omit/Readonly/Record`.
- 6 **Type Narrowing** — the Java `instanceof` pattern-matching parallel.
- 7 **Modules** — the same `export/import` as JavaScript, now type-checked.
- 8 **React with TypeScript** — converting real components/hooks/handlers from the React section into fully-typed code.

You now know how to add the SAME compile-time safety net you've always relied on in Java to your entire React frontend. Next: Redux Toolkit, covering state management for larger React applications.

(END OF TYPESCRIPT MASTER NOTES — ready to proceed to Redux Toolkit whenever you say "Next Topic.")